

[FOUNDER TOOLKIT] FINANCIAL MODELING TOOLKIT

Model the business / to understand it.

A financial model isn't a prediction — it's a **thinking tool** that forces you to make your assumptions about the business explicit. This is how to actually build one: the architecture, the drivers, and the discipline that makes investors believe it.

HOW TO READ THIS TOOLKIT

Three layers / from assumptions to decisions.

This is a build guide, not a theory lecture. Set up the **architecture** so the model is driver-based and the statements connect; build the **engine** bottoms-up from real unit economics; then make it **decision-grade** with scenarios and an honest cash story. It draws on the metrics from Business Operations and the cash discipline from Fundraising — here you build the actual model behind them.

LAYER 1

Set it up

The architecture

Structure & drivers

Why model at all · **the three statements** · driver-based design

LAYER 2

Build the engine

The engine

Revenue, cost, economics

Bottoms-up revenue · **cost & headcount** · unit economics

LAYER 3

Make it decide

Decision-grade

Scenarios & cash

Scenarios & sensitivity · **runway & the cash story** · the credible model

LAYER 1 • SET IT UP

The / architecture.

How you structure the model determines whether it's a living decision tool or a dead spreadsheet. Separate assumptions from calculations, connect the statements, and the model becomes something you can actually reason with.

Why model at all

The three statements

Driver-based design



TOOL 1.1 · WHY MODEL AT ALL

Why model / at all

Your projections will be wrong — that's fine. The model's value isn't the forecast; it's the rigorous thinking it forces about how the business actually works, and what has to be true for it to succeed.

— WHAT IT IS

A financial model is a **thinking tool first, a communication tool second**. Building it forces you to answer the questions that matter: how do we make money, how fast can we grow, when do we run out of cash, and which assumptions decide success or failure? You need one even if you never raise — the exercise surfaces the economics you'd otherwise hand-wave. The output is never precise; the **logic** is what's valuable. Treat it as a model of your *understanding* of the business, updated as you learn — not a promise you're making about the future.

— THE QUESTIONS IT ANSWERS

- ◇ What must be **true** for this business to work?
- ◇ How do we make money, grow, and when do we run out of cash?
- ◇ Which **few assumptions** drive success or failure?
- ◇ Is this a tool for **our** thinking, or just a deck prop?

— WHAT "GOOD" LOOKS LIKE

- ✓ The model clarifies the business, not just the pitch
- ✓ Built even absent fundraising — for your own decisions
- ✓ Logic valued over false precision
- ✓ Updated as you learn, not frozen at v1

— WHAT NOT TO DO — AND WHY

Building the model to impress, not to understand. Founders treat the model as a fundraising artifact — reverse-engineering a big number to justify the raise — instead of a tool to understand the business. The result looks polished and means nothing, and investors see through it. The opposite error is skipping the model because “it'll be wrong”

Patrick Connolly

— THE MODEL IS A THINKING TOOL

FORCES	How do we make money? Grow? Run out?
REVEALS	Which assumptions decide the outcome
OUTPUT	A forecast — always wrong
VALUE	The logic, not the number

■ AEGIS E0

Aegis built its model to answer real questions, not to dress a deck: how many DACH carriers at ~€30–50k each to reach default-alive, and what burn to get there. The exercise surfaced the assumption that **mattered most — pilot-to-paid conversion rate** — which then aimed the whole seed plan. The number on the last row was never the point; the **clarity about what had to be true** was.

TOOL 1.2 · THE THREE STATEMENTS

The three / statements

A complete model connects three views: the P&L (are we profitable?), the cash flow (will we survive?), and the balance sheet (what do we own and owe?). Cash flow is the one that keeps you alive.

— WHAT IT IS

The three statements answer different questions and **link together**. The **P&L** (income statement) shows revenue minus costs = profit over a period — distinguishing **COGS** (cost to deliver, e.g. hosting) from **OpEx** (salaries, rent, marketing), which sets your **gross margin**. The **cash-flow statement** tracks actual money in and out — the one that matters most early, because **startups die from running out of cash, not from accrued losses**. The **balance sheet** shows assets, liabilities, and equity at a point in time. Build them so they flow — assumptions → revenue → expenses → P&L → cash flow — and reconcile.

— THE QUESTIONS IT ANSWERS

- ◇ Does the model have all three statements, **linked**?
- ◇ Do we separate **COGS** from **OpEx** (so gross margin is real)?
- ◇ Are we tracking **cash**, not just accrued profit?
- ◇ Does the logic flow assumptions → revenue → P&L → cash?

— WHAT "GOOD" LOOKS LIKE

- ✓ Three connected statements, not just a P&L
- ✓ Clean COGS / OpEx split and a real gross margin
- ✓ Cash flow modeled — the survival statement
- ✓ Statements reconcile rather than contradict

— WHAT NOT TO DO — AND WHY

Modeling profit and ignoring cash. A P&L can look healthy while the bank account hits zero — timing mismatches (you pay salaries now, customers pay in 60 days) kill companies that are “profitable” on paper. Other classic errors: lumping COGS into OpEx so

— THREE LINKED STATEMENTS

P&L	Revenue – COGS – OpEx = profit
CASH FLOW	Money in / out — survival
BALANCE SHEET	Assets, liabilities, equity
LINK	They flow and reconcile

■ AEGIS E0

Aegis’s model kept the statements honest. Hosting and imagery-licensing costs sat in **COGS** (giving a real — healthy — software gross margin), while engineering and compliance salaries sat in **OpEx**. Critically, the **cash-flow view** captured the timing gap: carriers pay annually in arrears, so cash lagged signed revenue — the model showed exactly when the bank balance dipped, independent of the P&L.

TOOL 1.3 · DRIVER-BASED DESIGN

Driver-based / design

Put every assumption in its own labeled cell and reference it — never hardcode a number inside a formula. A driver-based model lets you change one input and watch the whole business respond.

— WHAT IT IS

A model is **driver-based** when its outputs are built from a small set of explicit **assumptions** (drivers) — conversion rate, churn, ARPU, hiring plan, sales-cycle length — each living in its own clearly labeled input cell. Formulas reference those cells (=customers*ARPU), never bury numbers inside (=850*49). This is the difference between a living tool and a dead spreadsheet: change one driver and the entire model recalculates, so you can ask “what if churn doubles?” instantly. Keep **inputs, calculations, and outputs visually separate** (a colour convention helps) so anyone can find and stress-test the assumptions.

— THE QUESTIONS IT ANSWERS

- ◇ Is every **assumption** in its own labeled input cell?
- ◇ Do formulas **reference drivers**, not hardcoded numbers?
- ◇ Can I change one input and see the **whole model** respond?
- ◇ Are inputs, calculations, and outputs **visually separated**?

— WHAT "GOOD" LOOKS LIKE

- ✓ A clear, isolated assumptions block driving everything
- ✓ No magic numbers buried inside formulas
- ✓ One input change flows through the entire model
- ✓ Inputs / calcs / outputs visually distinct

— WHAT NOT TO DO — AND WHY

Hardcoding assumptions inside formulas. Writing =850*49 instead of =customers*price turns the model into a black box: no one (including you, in a month) can find or change the assumptions, scenario analysis becomes impossible, and a single edit silently breaks the logic. Scattered, undocumented inputs make the model un-

— DRIVER-BASED, NOT HARDCODED

WRONG =850*49 — buried, un-auditable

RIGHT =customers*ARPU — references drivers

Assumptions in one block.

Change one cell → whole model moves.

■ AEGIS E0

Aegis's model had a single **assumptions tab**: carriers signed per quarter, pilot-to-paid conversion, ARPU band, churn, hires by month. Everything else was formulas referencing those drivers. So when an investor asked “what if conversion is half your estimate?”, Aegis changed **one cell** and the runway, revenue, and hiring plan all updated live — turning a tough question into a **two-second, credible answer**.

LAYER 2 · BUILD THE ENGINE

The / engine.

With the architecture set, build the substance: revenue from the bottom up, costs that scale realistically, and the unit economics that decide whether the business works at all. This is where a credible model is won or lost.

Bottoms-up revenue

Cost & headcount

Unit economics

2

TOOL 2.1 · BOTTOMS-UP REVENUE

Bottoms-up / revenue

Build revenue from the drivers you actually control — customers, conversion, price — not from a fantasy slice of a huge market. “We’ll capture 1% of a \$10B market” is the line investors are trained to dismiss.

— WHAT IT IS

Bottoms-up means constructing revenue from real, measurable mechanics: acquisition channels → conversion rates → customers → price/ARPU → revenue, layering in **churn** and **expansion** over time. For SaaS: Monthly revenue = customers × ARPU, with new customers from the funnel and retained customers net of churn (and mind the difference between **MRR, ARR, and recognised revenue**). Build it monthly for year 1, quarterly thereafter. The opposite — **top-down** (“1% of a \$10B TAM”) — skips all the real mechanics of growth and collapses on contact with reality. Ground every number in a driver you can defend.

— THE QUESTIONS IT ANSWERS

- ◇ Is revenue built from **funnel** → **customers** → **price**, bottoms-up?
- ◇ Are **churn and expansion** modeled, not just new sales?
- ◇ Do we distinguish **MRR / ARR / recognised revenue**?
- ◇ Can we **defend every driver** with data or a benchmark?

— WHAT "GOOD" LOOKS LIKE

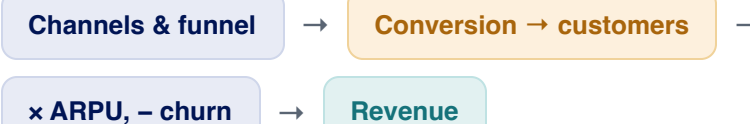
- ✓ Revenue built from controllable, measurable drivers
- ✓ Churn and expansion explicitly modeled over time
- ✓ MRR/ARR/revenue used correctly, not interchangeably
- ✓ Each assumption defensible, not aspirational

— WHAT NOT TO DO — AND WHY

Top-down market-share fantasy. “The market is \$10B; we’ll get 1%” is lazy and investors see right through it — it ignores conversion, customer behaviour, and how long acquisition actually takes. Equally damaging is the **hockey stick with no mechanics**: flat for six

Patrick Connolly

— BOTTOMS-UP, NOT TOP-DOWN



Never: “1% of a \$10B market.”
Every jump needs a mechanism.

■ AEGIS E0

Aegis modeled revenue bottoms-up from its actual motion: named DACH carriers in the pipeline × a defensible **pilot-to-paid conversion** × ~€30–50k ARPU, net of (low, sticky-enterprise) churn. No “1% of the European insurance market.” Every growth step tied to a concrete event — a carrier signing, a rep hired — so there were **no unexplained hockey-stick jumps** for an investor to puncture.

TOOL 2.2 · COST & HEADCOUNT

Cost / & headcount

For most startups, people are the biggest cost — so the hiring plan largely is the expense model.

Build costs from a real plan, and get which costs are fixed and which scale with growth right.

— WHAT IT IS

Build expenses from the **headcount plan** (the dominant cost for most startups) plus non-people OpEx. Tie hires to **milestones**, not a smooth curve — and remember **hiring takes time and ramps**, so a rep added in month 3 isn't productive in month 3. Classify costs correctly: **fixed** (rent, core salaries — steady regardless of volume) vs. **variable** (hosting, payment fees, support that scales with customers). Getting this wrong — e.g. assuming support headcount stays flat from 50 to 500 customers — makes the whole model lie. Start from current actual costs and add planned spend tied to growth.

— THE QUESTIONS IT ANSWERS

- ◇ Is the **hiring plan** explicit and tied to milestones?
- ◇ Have we accounted for hiring **lead time and ramp**?
- ◇ Which costs are **fixed** vs. **variable** with volume?
- ◇ Do costs start from **actuals**, then add planned spend?

— WHAT "GOOD" LOOKS LIKE

- ✓ Expenses driven by a real, staged hiring plan
- ✓ Hiring lead-time and ramp reflected, not instant
- ✓ Fixed vs. variable costs correctly classified
- ✓ Built up from current actuals

— WHAT NOT TO DO — AND WHY

Costs that don't move with the business. Two mirror errors: treating a variable cost as fixed (support headcount frozen as customers 10x — your margins are fiction) or a fixed cost as variable. Equally common: a hiring plan with no ramp, so revenue magically

— FIXED VS. VARIABLE

FIXED	Rent, core salaries — steady
VARIABLE	Hosting, fees, support — scale with volume
HIRING	Takes time + ramps — not instant

People are usually the biggest line.

■ AEGIS E0

Aegis's costs were mostly its small team, so the model *was* the hiring plan: the compliance/integration lead in month 4, a first commercial rep only after the motion was proven. Imagery licensing and hosting were modeled as **variable** (scaling with carriers); core salaries as **fixed**. The rep's revenue contribution was **ramped, not instant** — so the model didn't pretend a month-4 hire closed deals in month 4.

TOOL 2.3 · UNIT ECONOMICS

Unit / economics

Strip the business to a single customer: what does it cost to acquire one, and what are they worth? If the unit doesn't work, scaling just loses money faster — which is why investors look here first.

— WHAT IT IS

Unit economics ask whether **one customer** is profitable. The core pair: **CAC** (fully-loaded cost to acquire a customer — all sales & marketing ÷ customers won) and **LTV** (gross margin × average customer lifetime). Healthy benchmarks: **LTV:CAC ≥ 3:1**, **CAC payback < 12–18 months**, plus gross margin and retention (the same metrics as the Business Operations and Fundraising toolkits — here you build them from the model's drivers). Get these from your bottoms-up build, not a benchmark you borrowed: a competitor's churn rate is a sanity check, never a substitute for your own.

— THE QUESTIONS IT ANSWERS

- ◇ What's our fully-loaded **CAC** — all S&M ÷ customers won?
- ◇ What's **LTV** (gross margin × lifetime), and is LTV:CAC ≥ 3?
- ◇ Is **CAC payback** under 12–18 months?
- ◇ Are these from **our** drivers, or a borrowed benchmark?

— WHAT "GOOD" LOOKS LIKE

- ✓ CAC and LTV built from the model's own drivers
- ✓ LTV:CAC ≥ 3 and payback < 12–18 months
- ✓ Gross margin and retention modeled honestly
- ✓ Benchmarks used to sanity-check, not to fabricate

— WHAT NOT TO DO — AND WHY

Borrowing someone else's assumptions. Plugging in a competitor's 3% churn or a generic CAC because it makes the model look good — with no grounding in your own business — produces confident nonsense. The other trap is hiding bad unit economics behind top-line growth: if you lose money per customer, growth makes the loss *bigger*.

Patrick Connolly

≥3×

LTV : CAC

<12–18mo

CAC payback

70%+

gross margin (SaaS)

— THE UNIT

CAC (cost to acquire one)

vs

LTV (what one is worth)

■ AEGIS E0

Aegis's unit economics were strong precisely because the motion was efficient: CAC was low (warm referrals and pilots, not paid acquisition), ARPU ~€30–50k, gross margin high (software), and enterprise churn low — giving an **LTV:CAC well above 3 and fast payback**. Crucially these came from Aegis's *own* pipeline drivers, not a borrowed SaaS benchmark — which is exactly why the seed investors believed them.

LAYER 3 · MAKE IT DECIDE

Decision-grade.

A single-line forecast is a guess dressed up as a fact. A decision-grade model shows a range of futures, makes the cash story explicit, and survives scrutiny — which is what turns it from a deck prop into a tool you and your investors actually trust.

Scenarios & sensitivity

Runway & the cash story

The credible model

3

TOOL 3.1 · SCENARIOS & SENSITIVITY

Scenarios / & sensitivity

The future isn't one number. Model a base, an optimistic, and a pessimistic case — and find which assumptions the outcome is most sensitive to, so you know what to watch and what to de-risk.

— WHAT IT IS

Because every assumption is uncertain, model a **range**: a **base case** (your honest best estimate), an **optimistic** case, and a **pessimistic** one — the last being the one that actually protects you, since it shows what happens if things go slowly. Then run **sensitivity analysis**: change one driver at a time and see how much the outcome (runway, revenue) moves. This reveals the **few drivers that dominate** — usually conversion, churn, or sales-cycle length — so you know which assumptions to validate hardest and which risks to mitigate. A driver-based model (Layer 1) makes all of this a matter of changing inputs, not rebuilding.

— THE QUESTIONS IT ANSWERS

- ◇ Do we have **base, optimistic, and pessimistic** cases?
- ◇ Which **few drivers** move the outcome the most?
- ◇ Does the **pessimistic** case still leave us a path?
- ◇ Are we validating the **highest-sensitivity** assumptions hardest?

— WHAT "GOOD" LOOKS LIKE

- ✓ Three honest scenarios, not one hopeful line
- ✓ Sensitivity reveals the dominant drivers
- ✓ The downside case is realistic and planned for
- ✓ Effort focused on validating what matters most

— WHAT NOT TO DO — AND WHY

A single hopeful line presented as certainty. One smooth, optimistic projection hides all the risk and signals naivety to investors. Equally weak is a “conservative” case that’s just the base case minus 10% — not a real stress test. And running scenarios on a hardcoded

— THREE CASES + SENSITIVITY

BASE	Honest best estimate
OPTIMISTIC	Things go well
PESSIMISTIC	Slow — the one that protects you
SENSITIVITY	Which driver moves it most?

■ AEGIS E0

Aegis ran three cases off its driver tab. The variable everything hinged on was **pilot-to-paid conversion** — sensitivity analysis showed runway swung wildly with it, far more than with ARPU or hire timing. So Aegis knew exactly which assumption to validate hardest (convert Munich Re first) and sized the raise so that even the **pessimistic case left a viable path**, not a cliff.

TOOL 3.2 · RUNWAY & THE CASH STORY

Runway / & the cash story

The model's most important output is the cash balance over time: how long the money lasts, and what you'll have proven before it runs out. This is where the model meets the fundraising decision.

— WHAT IT IS

The cash-flow statement produces the number that decides the company's fate: **runway** — months until cash hits zero at the current burn. From the model you read **net burn** (monthly cash out minus in), the **cash-out date**, and, most importantly, **what milestones you'll have hit by then**. This connects straight to the Fundraising toolkit: size the raise to reach the **next round's milestones plus a buffer** (~18 months), and plan to start raising with **6+ months left**, never at the cliff. The model is what lets you say, credibly, "this raise gets us to *these* proof points with margin."

— THE QUESTIONS IT ANSWERS

- ◇ What's our **net burn** and our **cash-out date**?
- ◇ What **milestones** will we have hit before cash runs out?
- ◇ Does the raise fund **~18 months** to the next round + buffer?
- ◇ Will we start raising with **6+ months** of runway left?

— WHAT "GOOD" LOOKS LIKE

- ✓ Runway and cash-out date read directly from the model
- ✓ Milestones mapped against the cash timeline
- ✓ Raise sized to milestones + buffer (links to Fundraising)
- ✓ Next raise begun from strength, not the cliff

— WHAT NOT TO DO — AND WHY

Modeling revenue but never watching the cash balance. The fatal blind spot: a model that tracks growth and profit but doesn't surface the month the bank account goes negative. Founders discover the cliff too late and raise from desperation, which investors smell and price in. Equally, sizing a raise without a milestone-linked cash model means

Patrick Connolly

— THE CASH STORY

Net burn / month → Cash-out date → Milestones hit by then →

Size the raise

~18 months runway to next round.
Start raising at 6+ months left.

■ AEGIS E0

Aegis's model made the cash story explicit: net burn, the cash-out month, and the milestones due before then — convert Munich Re, sign two more carriers, harden compliance. It sized the seed to reach those **Series-A proof points with ~18 months runway plus buffer**, and the plan was to open the A conversation with **months of runway still in the bank** — exactly the from-strength position the Fundraising toolkit demands.

TOOL 3.3 · THE CREDIBLE MODEL

The credible / model

A model investors believe is one where every number traces to a defensible driver, the figures are consistent everywhere, and the assumptions are ambitious but grounded. Credibility, not optimism, is what raises money.

— WHAT IT IS

Everything converges here: a **credible** model is driver-based and auditable (every number traces to an assumption), **internally consistent** (the model matches the deck matches the data room — no \$2M-vs-\$1.5M contradictions), **bottoms-up**, with **defensible assumptions** (ambitious but grounded in real data or honest benchmarks) and a visible **range**. Investors don't expect accuracy — they expect rigour: they're testing whether you understand your business deeply enough to be trusted with capital. Keep it living — track **actuals vs. forecast** — so it stays a decision tool, not a frozen artifact.

— THE QUESTIONS IT ANSWERS

- ◇ Can every number be **traced to a defensible driver**?
- ◇ Do the model, deck, and data room **agree** exactly?
- ◇ Are assumptions **ambitious but grounded**, not fantasy?
- ◇ Do we track **actuals vs. forecast** and update it?

— WHAT "GOOD" LOOKS LIKE

- ✓ Every figure traceable to an assumption
- ✓ Consistent numbers across model, deck, and data room
- ✓ Assumptions defensible under questioning
- ✓ A living model, tracked against actuals

— WHAT NOT TO DO — AND WHY

Polished optimism that collapses under questions. A beautiful model with numbers that don't reconcile (deck says one thing, model another), assumptions you can't defend, or a hockey stick with no mechanism destroys trust the instant diligence begins — investors read it as either naivety or spin. The fix isn't more conservative numbers; it's rigour:

— WHAT MAKES IT CREDIBLE

TRACEABLE	Every number → a driver
CONSISTENT	Model = deck = data room
GROUNDING	Ambitious but defensible
LIVING	Actuals vs. forecast, updated

■ AEGIS E0

Aegis's model survived diligence because it was built for it: driver-based and auditable, every figure tracing to the assumptions tab, and **perfectly consistent with the deck and data room** (the same numbers everywhere, per the Legal toolkit's diligence discipline). Assumptions were ambitious but grounded in the 52-conversation evidence base. Investors weren't buying the forecast — they were buying the **visible rigour behind it**.

PUTTING IT TOGETHER

The model is / how you think.

A financial model earns its keep not as a forecast but as a discipline.

Architect it so assumptions drive everything; build the engine bottoms-up from real unit economics; make it decision-grade with scenarios and an honest cash story. The output will always be wrong — but a rigorous, living model is how you understand your business, and how investors decide they can trust you with it.

Layer 1

Architecture

A thinking tool, three linked statements, and a driver-based design you can stress-test by changing one cell.

Layer 2

Engine

Revenue bottoms-up, costs that scale realistically, and unit economics built from your own drivers.

Layer 3

Decide

Scenarios and sensitivity, an explicit cash story, and a credible, consistent, living model.

The point

Rigour

Investors don't expect accuracy — they expect rigour. The model proves you understand your business.

THE EVIDENCE BASE

Sources / & further reading.

This toolkit is a build guide grounded in established financial-modeling practice and named expert sources (Christoph Janz, David Skok). It deliberately connects to the other toolkits — the unit-economics metrics from Business Operations, the cash and runway discipline from Fundraising, and the consistency standard from Legal & Incorporation — and shows how to build the actual model behind them. Aegis EO is a fictional teaching venture; its figures are illustrative.

The model as a thinking tool

Fractional CFO practice: a model is a thinking tool first, a communication tool second — the value is the rigour, not the forecast.

Driver-based modeling

Modeling best practice (EY startup guide; FP&A standards): assumptions in isolated input cells, formulas reference drivers, never hardcode.

The SaaS financial model

Christoph Janz (Point Nine Capital): the widely-used SaaS plan template — MRR/ARR, pricing tiers, churn; distinguishing MRR, ARR, and revenue.

Cost & headcount

CFO practice: people are the dominant cost; tie hires to milestones with ramp; classify fixed vs. variable costs correctly.

Common modeling mistakes

CFO Bridge; Inflection CFO: the hockey stick with no mechanics, hardcoded assumptions, 100% gross margins, fixed-vs-variable errors, borrowed benchmarks.

The three statements

Standard financial practice: linked P&L, cash flow, and balance sheet; COGS vs. OpEx; startups die from cash, not accrued losses.

Bottoms-up revenue

EY; CFO practice: build revenue from funnel → conversion → customers → ARPU; reject top-down “1% of a \$10B market” projections.

Unit economics

David Skok (*For Entrepreneurs*): CAC, LTV, LTV:CAC ≥ 3 , CAC payback < 12–18 months — the economics that decide whether to scale.

Scenarios & sensitivity

Modeling standards: base / optimistic / pessimistic cases; sensitivity analysis to find the drivers that dominate the outcome.

Runway & the raise

Connects to the Fundraising toolkit: size the raise to ~18-month milestones + buffer; start raising with 6+ months of runway left.